

ui.notification 全面详解

ui.notification 是 NiceGUI 提供的**可动态更新**通知组件，基于 Quasar Notify API 封装，核心用于显示可实时修改内容、状态的临时消息提示（如进度展示、异步操作反馈、动态状态更新等）。与简洁的 `ui.notify` 相比，它支持显示后修改消息、切换状态、控制加载动画等高级功能，是复杂交互场景中反馈用户操作的核心组件。

一、核心概念与基础特性

1. 本质与用途

- 本质：实例化通知组件，通过类属性和方法动态控制通知的内容、样式、状态，无需重复创建即可实现多阶段反馈。
- 核心用途：适用于需要持续更新状态的场景，常见场景包括：
 - 进度展示：如“文件上传 30%”“数据处理 75%”；
 - 异步操作多阶段反馈：如“连接服务器→处理数据→完成”；
 - 动态状态提示：如“网络连接中→连接成功”“任务排队中→执行中→完成”；
 - 持久化可交互通知：如带关闭按钮、状态切换的重要提示。
- 关键机制：
 - 实例化后自动显示（无需手动调用显示方法），默认 5 秒后自动关闭（可配置超时或禁用自动关闭）；
 - 支持实时修改消息文本、图标、颜色、类型、加载状态等属性；
 - 提供 `dismiss()` 方法手动关闭通知，关闭时可触发回调函数；
 - 支持显示加载动画（spinner），适配异步操作等待场景。

2. 基础结构

ui.notification 需通过**实例化**使用，基础示例如下，核心展示动态更新能力：

```
from nicegui import ui
import asyncio

# 动态更新通知内容和状态
async def progress_demo():
    # 初始化通知：无超时（手动关闭）、显示加载动画、居中显示
    notification = ui.notification(
        message='开始处理...',
        type='info',
        spinner=True,
        timeout=None,
        position='center',
        close_button='取消' # 显示关闭按钮，标签为“取消”
    )

    # 模拟进度更新（10 个阶段）
    for i in range(1, 11):
        await asyncio.sleep(0.3)
        # 动态修改消息（显示进度）
```

```
notification.message = f'处理进度: {i * 10}%'
# 进度 100% 时切换状态和样式
if i == 10:
    notification.message = '处理完成!'
    notification.type = 'positive' # 成功状态（绿色）
    notification.spinner = False # 隐藏加载动画
    notification.timeout = 2 # 2 秒后自动关闭

# 触发通知
ui.button('启动动态通知', on_click=progress_demo).classes('mt-4')

ui.run()
```

二、初始化配置项

实例化 `ui.notification(**kwargs)` 时可通过参数配置初始状态，参数说明如下（基于官方文档完整配置）：

参数名	类型	说明
message	str	通知初始内容（必填，支持纯文本）
position	str	显示位置（默认 bottom），支持值：top-left / top-right / bottom-left / bottom-right / top / bottom / left / right / center
close_button	bool str	是否显示关闭按钮（默认 False），传字符串时为按钮标签（如 '关闭'）
type	str	通知类型（默认 None），支持值：positive（成功）、negative（错误）、warning（警告）、info（信息）、ongoing（进行中）
color	str	自定义颜色（优先级高于 type，支持 Quasar 颜色名如 'blue' 或十六进制色 '#FF5733'）
multi_line	bool	是否支持多行文本（默认 False），开启后自动换行
icon	str	左侧图标（默认 None），支持 Quasar 图标库名称（如 'check_circle'）
spinner	bool	是否显示加载动画（默认 False），适用于异步等待场景
timeout	float None	自动关闭超时时间（单位：秒，默认 5.0），设为 None 时永不自动关闭
on_dismiss	Callable	通知关闭时触发的回调函数（无参数或接收 UiEventArguments 参数）
options	dict	可选字典，覆盖所有其他参数（如 options={'message': '内容', 'timeout': 3}）
classes	str	CSS 类名（支持 Tailwind、Quasar 类，如 'bg-gray-100'）
props	str	Quasar 组件属性（如 'rounded-lg' 圆角、'shadow-md' 阴影）
style	str	内联 CSS 样式（如 'font-size: 14px;'）

配置示例（多参数组合）

```
from nicegui import ui

def on_notification_dismiss():
    ui.notify('通知已关闭', type='info')

# 多配置组合示例
ui.button('复杂配置通知', on_click=lambda: ui.notification(
    message='重要提示：请及时保存数据',
    type='warning',
    icon='warning',
    position='top-right',
    close_button='知道了',
    timeout=10.0, # 10 秒后自动关闭
    on_dismiss=on_notification_dismiss,
    classes='bg-yellow-50 text-yellow-800',
    props='rounded-xl shadow-lg'
)).classes('mt-4')

ui.run()
```

三、核心属性（可动态修改）

ui.notification 实例的核心属性支持**动态赋值修改**，无需重新创建即可更新界面，关键属性如下：

属性名	类型	说明
message	str（可设置）	通知内容，动态修改后实时更新界面
type	str None（可设置）	通知类型，支持动态切换（如从 info 改为 positive）
color	str None（可设置）	自定义颜色，修改后覆盖类型默认颜色
icon	str None（可设置）	左侧图标，支持动态切换或隐藏（设为 None 隐藏）
spinner	bool（可设置）	是否显示加载动画，动态控制显示 / 隐藏
timeout	float None（可设置）	自动关闭超时时间，修改后立即生效（如从 None 改为 3.0）
close_button	bool str（可设置）	动态控制关闭按钮（如从 False 改为 '关闭'）
position	str（可设置）	动态修改显示位置（如从 bottom 改为 center）
multi_line	bool（可设置）	动态开启 / 关闭多行文本支持
visible	BindableProperty	通知可见性（布尔值，支持双向绑定，True 显示、False 隐藏）
html_id	str	HTML DOM 中的元素 ID（版本 2.16.0 新增，只读）
is_deleted	bool	元素是否已删除（只读）

属性动态修改示例

```
from nicegui import ui
import asyncio

async def dynamic_update_demo():
    notification = ui.notification(
        message='初始状态: 信息通知',
        type='info',
        timeout=None,
        position='center'
    )

    # 阶段 1: 切换为警告类型
    await asyncio.sleep(2)
    notification.message = '状态更新: 警告提示'
    notification.type = 'warning'
    notification.icon = 'warning'

    # 阶段 2: 切换为成功类型, 隐藏图标
    await asyncio.sleep(2)
    notification.message = '状态更新: 操作成功'
    notification.type = 'positive'
    notification.icon = None
    notification.spinner = False

    # 阶段 3: 自定义颜色, 添加关闭按钮
    await asyncio.sleep(2)
    notification.message = '状态更新: 自定义颜色'
    notification.color = 'purple' # 覆盖成功类型颜色
    notification.close_button = '关闭'

# 触发演示
ui.button('动态修改属性', on_click=dynamic_update_demo).classes('mt-4')

ui.run()
```

四、核心方法

ui.notification 实例提供丰富方法用于控制生命周期和状态，常用方法如下：

方法名	作用	示例
dismiss()	手动关闭通知（触发 on_dismiss 回调）	<code>notification.dismiss()</code>
on_dismiss(callback)	动态绑定关闭回调（覆盖初始化时的 on_dismiss）	<code>notification.on_dismiss(lambda: ui.notify('已关闭'))</code>
set_visibility(visible: bool)	控制通知可见性（True 显示、False 隐藏）	<code>notification.set_visibility(False)</code>

方法名	作用	示例
update()	强制更新客户端界面（修改属性后可选调用，确保同步）	<code>notification.update()</code>
delete()	永久删除通知元素（无法恢复）	<code>notification.delete()</code>
tooltip(text: str)	为通知添加悬浮提示（较少用）	<code>notification.tooltip('点击关闭')</code>

方法使用示例

```
from nicegui import ui

def method_demo():
    # 初始化通知
    notification = ui.notification(
        message='可手动控制的通知',
        type='info',
        timeout=None,
        position='bottom-right'
    )

    # 动态绑定关闭回调
    notification.on_dismiss(lambda: ui.notify('通知被手动关闭'))

    # 外部按钮控制通知
    def close_notification():
        notification.dismiss() # 手动关闭

    def hide_notification():
        notification.set_visibility(False) # 隐藏通知

    def show_notification():
        notification.set_visibility(True) # 显示通知

    # 控制按钮
    ui.row(
        ui.button('关闭通知', on_click=close_notification),
        ui.button('隐藏通知', on_click=hide_notification),
        ui.button('显示通知', on_click=show_notification)
    ).classes('mt-2')

ui.button('创建可控制通知', on_click=method_demo).classes('mt-4')

ui.run()
```

五、高级用法（实战场景）

1. 进度条整合（复杂内容展示）

结合 NiceGUI 组件实现带进度条的通知，适用于文件上传、数据处理等场景：

```
from nicegui import ui
import asyncio

async def progress_bar_demo():
    # 创建通知实例（无内容，后续动态添加组件）
    notification = ui.notification(
        message='', # 空消息，用组件替代
        timeout=None,
        position='center',
        classes='p-4 bg-white rounded-xl shadow-lg'
    )

    # 向通知中添加组件（进度条+文本）
    with notification:
        message_label = ui.label('文件上传中...').classes('font-medium')
        progress_bar = ui.progress(value=0).classes('w-64 mt-2')

    # 模拟上传进度
    for i in range(1, 101):
        await asyncio.sleep(0.05)
        progress_bar.value = i / 100
        message_label.text = f'文件上传中: {i}%'

    # 上传完成后更新内容
    message_label.text = '文件上传完成!'
    progress_bar.value = 1.0
    notification.type = 'positive'
    notification.timeout = 2 # 2 秒后关闭

ui.button('上传文件演示', on_click=progress_bar_demo).classes('mt-4')

ui.run()
```

2. 异步操作错误处理（多状态切换）

适配异步任务的成功 / 失败分支，动态切换通知状态和内容：

```
from nicegui import ui
import asyncio

async def async_task_demo():
    # 初始化加载通知
    notification = ui.notification(
        message='请求接口中...',
        spinner=True,
        type='info',
```

```

        timeout=None,
        position='center'
    )

    try:
        # 模拟异步接口请求（3 秒）
        await asyncio.sleep(3)
        # 模拟随机成功/失败
        import random
        if random.choice([True, False]):
            # 成功状态
            notification.message = '接口请求成功!'
            notification.type = 'positive'
            notification.spinner = False
            notification.timeout = 2
        else:
            # 失败状态
            notification.message = '接口请求失败：网络超时'
            notification.type = 'negative'
            notification.spinner = False
            notification.close_button = '重试'
            # 绑定关闭按钮点击事件（重试逻辑）
            notification.on_dismiss(lambda: async_task_demo())
    except Exception as e:
        # 异常处理
        notification.message = f'错误: {str(e)}'
        notification.type = 'negative'
        notification.spinner = False
        notification.close_button = '关闭'

    ui.button('执行异步任务', on_click=async_task_demo).classes('mt-4')

    ui.run()

```

3. 全局通知管理（统一控制多个通知）

通过列表管理多个通知实例，实现批量关闭、统一样式等功能：

```

from nicegui import ui
import asyncio

# 存储通知实例的列表
notifications = []

async def create_notification(index):
    # 创建通知并加入列表
    n = ui.notification(
        message=f'通知 {index}: 正在运行',
        type='ongoing',
        timeout=None,
        position='bottom-right'
    )

```

```
notifications.append(n)

# 模拟运行 5 秒
await asyncio.sleep(5)
n.message = f'通知 {index}: 已完成'
n.type = 'positive'
n.timeout = 1

# 批量创建通知
def batch_create():
    for i in range(3):
        ui.timer(i * 0.5, lambda idx=i+1: asyncio.create_task(create_notification(idx)))

# 批量关闭所有通知
def batch_dismiss():
    for n in notifications:
        if not n.is_deleted:
            n.dismiss()
    notifications.clear()

# 控制按钮
ui.row(
    ui.button('批量创建通知', on_click=batch_create),
    ui.button('批量关闭通知', on_click=batch_dismiss)
).classes('mt-4')

ui.run()
```

六、与 ui.notify 的核心区别

特性	ui.notification	ui.notify
调用方式	实例化 (ui.notification())	函数式 (ui.notify())
动态更新	支持 (修改实例属性即可)	不支持 (创建后无法修改内容 / 状态)
核心场景	进度展示、多阶段反馈、动态状态更新	简单操作结果提示、一次性消息
高级配置	支持加载动画、关闭回调、动态绑定	仅支持基础样式和位置配置
生命周期控制	实例方法控制 (dismiss() / update())	自动关闭或手动关闭 (无实例控制)

七、注意事项

- 超时单位: `timeout` 参数单位为秒 (如 `timeout=3` 表示 3 秒) , 与 `ui.notify` 的毫秒单位不同, 需注意区分。
- 动态更新生效: 修改 `message`、`type` 等属性后, 界面会自动更新, 无需手动调用 `update()`; 若修改 `classes`、`props` 等样式属性, 建议调用 `update()` 确保同步。
- 关闭按钮逻辑: `close_button` 设为字符串时, 点击按钮会触发 `dismiss()` 并关闭通知, 无需额外绑定事件。

4. 多_line 换行: 开启 `multi_line=True` 后, 文本会自动换行; 若需手动换行 (如 `\n`), 需配合 CSS 样式 `white-space: pre-line` (可通过 `classes` 配置)。
5. 版本兼容性: `timeout` 属性支持动态修改需 NiceGUI 2.13.0+ 版本; `html_id` 属性需 2.16.0+ 版本, 使用时需确认版本匹配。
6. 性能优化: 避免同时创建大量实例化通知, 可能导致界面卡顿; 短时间内多次更新属性时, 建议控制更新频率 (如通过 `ui.timer` 批量更新)。

通过以上配置与方法, `ui.notification` 可灵活满足复杂场景下的动态反馈需求, 尤其适合需要持续更新状态的异步操作、进度展示等场景, 是 NiceGUI 中功能最强大的通知组件之一。